

IpePlots – User Manual with Examples

Jan Hlavacek
(jhlavace@svsu.edu)

June 26, 2025

Contents

1	Introduction	1
2	Installation	2
3	Usage	3
3.1	Coordinate System	3
3.2	Coordinate Grid	4
3.3	Parametric Plot	6
3.4	Function Plot	7
3.5	Data Plot	8
4	Examples	12
4.1	Plotting a Piecewise Function	12
4.2	Function with Vertical Asymptote	18
4.3	Trigonometric Plot	20

Abstract

IpePlots is an extension (so called “ipelet”) for the graphics editor IPE [2]. The purpose of this extension is to make creation of plots of functions, especially the type of plots used in mathematics education, easier. We provide basic introduction to IpePlots, as well as several step by step examples.

1 Introduction

The IPE graphics editor, written by Otfried Cheong, is a drawing editor for creating figures in PDF or encapsulated PostScript format. According to the Ipe website [2], its main features are:

- Entry of text as $\text{\LaTeX}$ source code. This makes it easy to enter mathematical expressions, and to reuse the $\text{\LaTeX}$ -macros of the main document. In the display text is displayed as it will appear in the figure.

- Produces pure Postscript/PDF, including the text. IPE converts the $\text{\LaTeX}$ -source to PDF or Postscript when the file is saved.
- It is easy to align objects with respect to each other (for instance, to place a point on the intersection of two lines, or to draw a circle through three given points) using various snapping modes.
- Users can provide ipelets (IPE plug-ins) to add functionality to IPE. This way, IPE can be extended for each task at hand.
- IPE can be compiled for Unix and Windows.
- IPE is written in standard C++ and Lua 5.1.

IpePlots is a plotting extension for the IPE. Its can help you include plots of functions, tabulated data, parametric curves and coordinate systems into your IPE drawings. IpePlots is written in Lua 5.1. It is released under the GPL v.2.0.

2 Installation

We will assume that you already have the IPE graphics editor installed on your computer. You can obtain the editor from the IPE webpage [2].

Installation of IpePlots is very simple. All you have to do is to place the file `plots.lua` in the “ipelets” directory on your computer. On Unix and Unix-like systems, you can use for example the `$HOME/.ipe/ipelets` directory. On Windows, the directory is determined by the value of the `IPELETPATH` environment variable. After installing IpePlots, simply restart IPE. If the installation was successful, you will have a “Plots” sub-menu under IPE’s “Ipelets” menu (Figure 1).

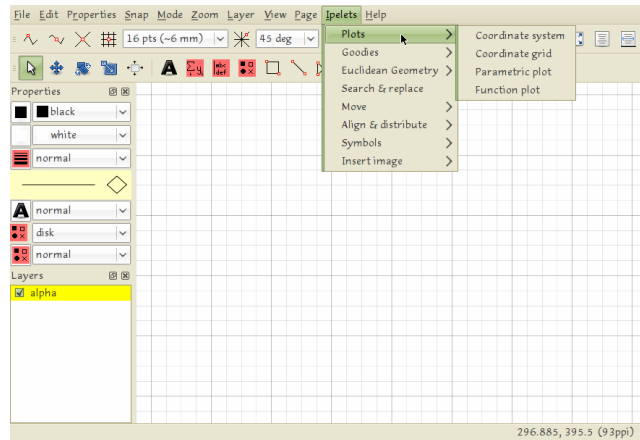


Figure 1: IPE window with the “Plots” sub-menu open

3 Usage

To insert a plot or a coordinate system into your drawing, select one of the items in the “Plots” sub-menu under the “Ipelets” menu. In the current version of IpePlots, there are four items:

Coordinate system will insert a Cartesian coordinate system into your drawing. It will consist of the horizontal axis, the vertical axis, and optional ticks. IpePlots currently does not create any labels, if you want to label the axes or ticks, you have to do so manually.

Coordinate grid will insert a rectangular grid of vertical and horizontal line segments into your drawing. You can specify the location of the segments.

Parametrid plot will insert a parametric curve defined by two functions, $x = f(t)$ and $y = g(t)$.

Function plot will insert a plot of a function $y = f(x)$.

Data plot will plot your data saved in a data file (.csv or alike).

Depending on several things, IpePlots will do one of the following:

- If your current selection has a non-empty bounding box, IpePlot will use this bounding box as a “viewport” which will contain the coordinate system of the plot.
- If you do not have a current selection, or if the selections bounding box is empty, IpePlot will use the canvas coordinate system in the following way:
 - If you previously set the origin of the IPE axis system, it will be used as the origin of the plot coordinate system. The base direction of the axis system is ignored, however, and the plot axis are always created horizontal and vertical.
 - If you did not set the origin of the axis system, the absolute canvas coordinates are used instead.

After selecting one of the menu items, you will be presented with a dialog box.

The current version of IpePlots provides the following four menu items:

3.1 Coordinate System

creates a pair of coordinate axes, with optional ticks. If the current selection has a non-empty bounding box, the axes will be scaled so that they will exactly fit inside this bounding box. In the dialog box (Figure 2) you can set the range for x , the range for y , the optional size of ticks (defaults to 0 for no ticks), and the location of ticks (if left empty, and the size is non-zero, ticks are placed at integer coordinates).

Set coordinates for viewport:

from x= to x=

from y= to y=

Size of x-ticks (in pt): Size of y-ticks (in pt):

Locations of x-ticks:

Locations of y-ticks:

Figure 2: Dialog box for the Coordinate System

Note that in all fields except the two tick size fields, you can use Lua expressions, which makes it possible to enter values like `-pi - 0.2` or `sqrt(3)/2`.

The syntax for the location of ticks is special: you could specify a comma separated list of numbers, or you could enter a single Lua table containing numbers. That way you can enter a Lua expression that generates a table of numbers. For example, IpePlots provides an internal function `range(from, to, step)` which produces a table of numbers starting with the value of “from” and incrementing by “step” until it exceeds “to”. The values entered in the Figure 2 will produce the coordinate system in Figure 3. Note that the rectangle containing the coordinate system was not created by IpePlots. It was already present in the drawing, and we selected it before using IpePlots. The coordinate system was created by IpePlots in such a way that it fits perfectly inside the bounding box of the rectangle. The rectangle can be deleted after the coordinate system is created.

Also note that IpePlots does not create labels. We see that as an advantage. IpePlots will quickly create an axes system for you, and you can then label it in any way you want, IpePlots will not impose any labeling style on you. IPE’s vertex snapping mode and the “align” and “move” ipelet groups can be very helpful when adding labels.

If your selection is empty, or has an empty bounding box, you will be presented with the same dialog box, however, instead of scaling the coordinates in order to fit them into the given bounding box, IpePlots will use the absolute canvas coordinates. That means a coordinate system from $-\pi$ to π would be really small, so it is probably more useful to specify coordinates like -50 to 50 .

3.2 Coordinate Grid

will create a rectangular grid of horizontal and vertical line segments at specified coordinates. The dialog box (see Figure 4) is very similar to the dialog box for Coordinate System, except that there are no fields for tick size, and instead of tick locations, you specify the locations of vertical and horizontal grid lines.

The coordinate grid created by IpePlots with the values filled in as in Figure 4

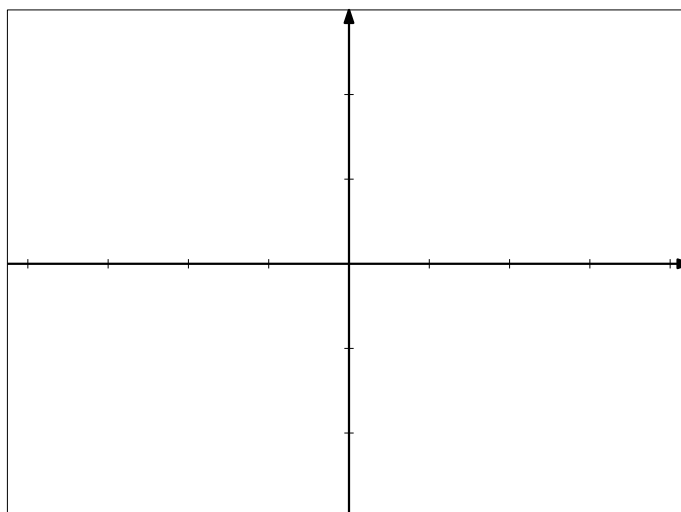


Figure 3: An example of a coordinate system produced by IpePlots.

Set coordinates for viewport:			
from x=	-pi-.2	to x=	pi+.2
from y=	-3	to y=	3
Locations of vertical grid lines:	range(-pi,pi,pi/4)		
Locations of horizontal grid lines:	-2, -1, 1, 2		
		Cancel	Ok

Figure 4: Dialog box for the Coordinate System

will produce the coordinate grid in Figure 5. Again, the rectangle containing the coordinate grid was not created by IpePlots. IpePlots created the coordinate grid inside the existing rectangle.

Note that only the coordinate grid was created, not the coordinate axes. If you want to create both, you have to use both “Coordinate System” and “Coordinate Grid” items from the “Plots” menu. You can find some examples of a complete work flow in section 4 on page 12.

As for Coordinate System, if the current selection is empty or has an empty bounding box, IpePlot will use the absolute canvas coordinates when creating the grid.

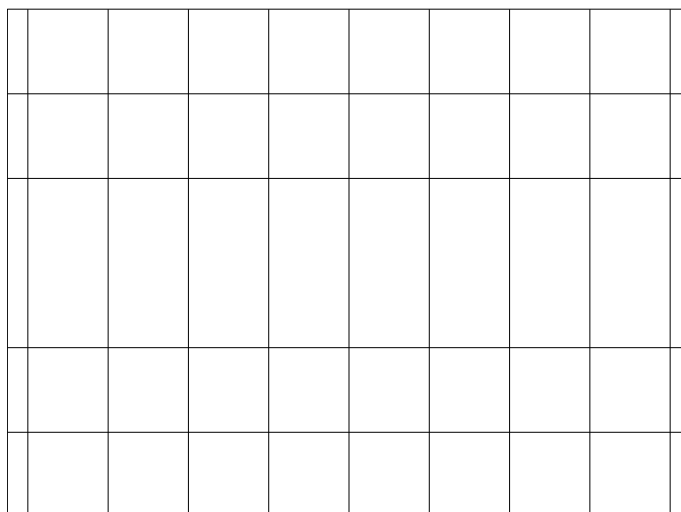


Figure 5: An example of a coordinate grid produced by IpePlots.

3.3 Parametric Plot

creates a plot of a curve described by the parametric equations

$$\begin{aligned}x &= f(t) \\ y &= g(t) \\ a &\leq t \leq b\end{aligned}$$

The dialog box for Parametric Plot is shown in the Figure 6. You need to specify x and y as functions of a parameter t , and the bounds for t . If the current selection has a non-empty bounding box, you also have to specify the ranges of x and y coordinates that correspond to the bounding box. The plot will be scaled in such a way that the given x and y coordinate ranges will fit exactly into the bounding box.

You also need to specify the number of points used to draw the plot. The more points you use, the more precise is your plot going to be. On the other hand, the more points you specify, the larger the file containing the drawing, and with large number of points, IPE may slow down significantly, especially on systems with low resources. The default of 100 seems to generally be a reasonable compromise.

Finally, you can select whether you want the plot to be approximated by cubic splines or by series of line segments. Cubic splines will generally produce a smoother curve. Note that if you have less than 4 points specified, the cubic spline option will be ignored.

The plot generated from the values entered in Figure 6 is shown in Figure 7. Again, the rectangle containing the Lissajous curve was not generated

Enter parametric equations. Use t as a parameter.

x=

y=

Set the domain for t:

from: to:

Set coordinates for viewport:

from x= to x=

from y= to y=

number of points

☒ use cubic splines

Figure 6: Dialog box for the Parametric Plot

by IpePlots. It was used as a bounding box, which will exactly represent the coordinate rectangle $-1.2 \leq x \leq 1.2$, $-1.2 \leq y \leq 1.2$.

If the current selection is empty or has an empty bounding box, there is no need to specify the ranges of x and y coordinates, since IpePlots will be using the absolute canvas coordinates. The dialog box presented to you in such a case will not have the fields for these coordinates. You can use this mode to insert precise curves in the absolute canvas coordinates into your drawing. For example, the ornamental curve in Figure 8 was created by combining a partial Lissajous curve in absolute canvas coordinates with two semicircles.

3.4 Function Plot

creates a graph of a function $y = f(x)$. Note that IpePlots has no special treatment for things like discontinuities, asymptotes etc. See the section 4.2 on page 18 to see an example of plotting a graph of a function with a vertical asymptote.

The dialog box for the Function Plot is shown in Figure 9. In this dialog, you need to enter the actual function, the domain over which the function should be graphed, and the coordinate limits for the bounding box. If the current selection is empty or has an empty bounding box, the limits for the bounding box will not be present in the dialog. Just as in the parametric plot dialog, you can change the number of points used to draw the graph, and choose whether you want to use line segments or cubic splines to approximate the curve.

The graph created from the parameters entered in Figure 9 is shown in Figure 10. As before, the rectangle containing the graph was not created by IpePlots, instead it was used as a bounding box to fit the graph into.

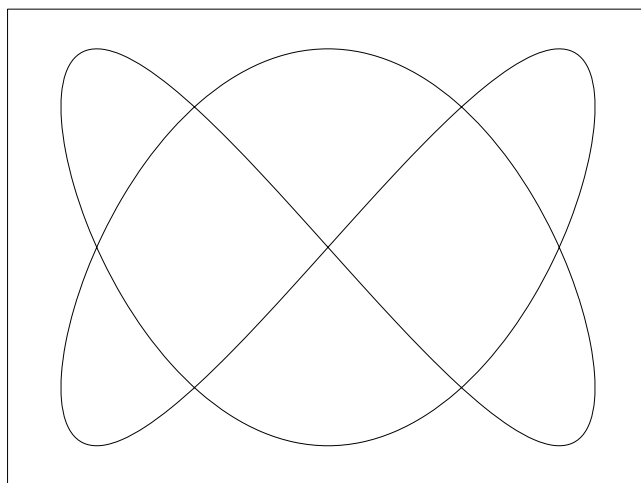


Figure 7: An example of a parametric plot produced by IpePlots.

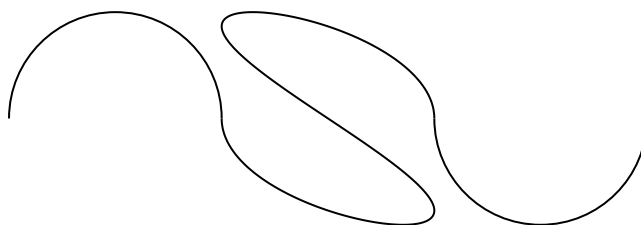


Figure 8: An example of a curve created by combining a parametric curve in absolute canvas coordinates with two semicircles.

3.5 Data Plot

creates a graph of your data stored in a data file of a `.csv` type. IpePlots assume that the file contains data arranged to columns such that one (not necessarily the first) column stores the x data and another (not necessarily the second) column stores the y data.

The dialog box for the Data Plot is shown in Figure 11. In this dialog, you first need to specify the data source. Two file types were originally considered:

- `.csv` file – columns are separated by commas → use “,” as the column separator;
- data files where columns are separated by blank characters, such as white-space or tab → use “%s” as the column separator.

If the data file contains commented lines that you want IpePlot to skip, you can specify one (!) character that marks the beginning of the comment. You also

Enter y as a function of x

y=

Set the domain for x:

from: to:

Set coordinates for viewport:

from x= to x=

from y= to y=

number of points

☒ use cubic splines

Figure 9: Dialog box for the Function Plot

have to specify which column contains the x data and which column contains the y data.

The viewport setting is the same as in the previous plot types. If the current selection is empty or has an empty bounding box, the limits for the bounding box will not be present in the dialog.

Finally, you can choose if you want to plot your data as points only, as connected points or as lines or splines. If the box **connect points** is checked, the connections between adjacent data points will be drawn as straight lines (if **use cubic splines** is NOT checked) or as splines (if **use cubic splines** IS checked). If the box **connect points** is not checked, (un)checking the box **use cubic splines** has no effect. Note also that cubic splines may not be a good approximation of your data, especially when they are not reasonably equally spaced. If the box **use symbols** is checked, data will be plotted as symbols. You can choose the symbol type from you current stylesheet(s) using the drop-down list. The symbol type selected globally in the IPE side panel does not have any effect during creation of the Data Plot, but can affect the plot once it has been created.

All objects created at once are grouped together in an IPE group. You can ungroup them from the drop-down menu when right-clicking on the created plot. If you, for example, want to have one group for data points and another for the connecting polyline/spline, you must create them separately by calling Data Plot twice.

The graph created from the parameters entered in Figure 11 is shown in Figure 12. As before, the rectangle containing the graph was not created by IpePlots, instead it was used as a bounding box to fit the graph into.

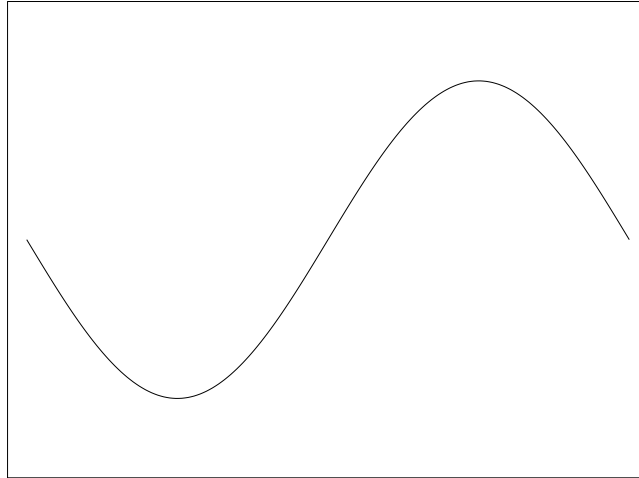


Figure 10: An example of a graph of a function produced by IpePlots.

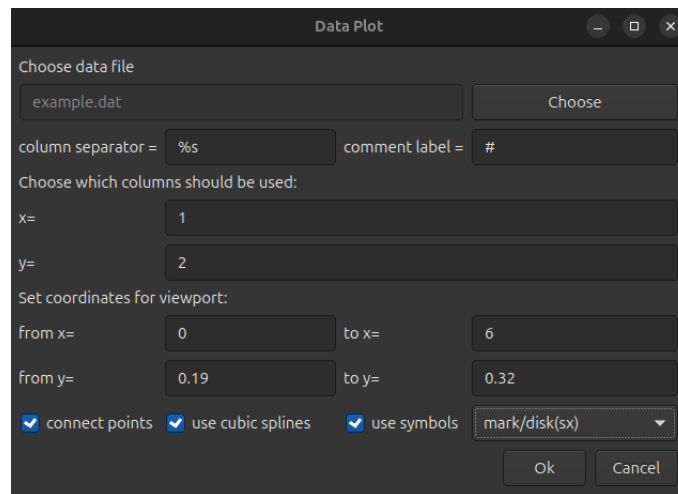


Figure 11: Dialog box for the Data Plot

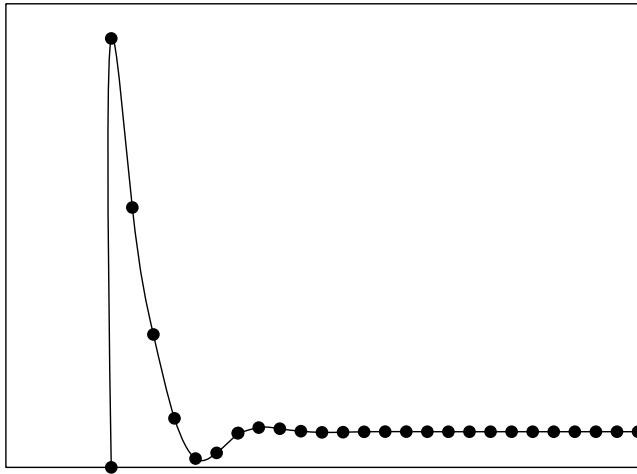


Figure 12: An example of a graph of data produced by IpePlots (data source `example.dat`).

4 Examples

4.1 Plotting a Piecewise Function

In this example we will create a plot of the piecewise defined function:

$$f(x) = \begin{cases} x + 5 & \text{if } x \leq -2 \\ x^2 - 1 & \text{if } -2 \leq x < 1 \\ 2 - x & \text{if } x \geq 1 \end{cases}$$

To show all the features, we will create a viewing rectangle approximately $-5 < x < 5$ and $-4 < y < 4$.

In order to create a plot with a 1 : 1 aspect ratio, we need start by creating a rectangle with the ratio of horizontal to vertical side 5 : 4. We can conveniently use the grid snapping mode in IPE(see the IPE manual([1]) for details). An example of such rectangle is shown in the Figure 13 Notice that the rectangle is shown in **red**, which means that it is currently selected.



Figure 13: A rectangle with 5 : 4 side ratio

Making sure the rectangle is selected, choose the “Coordinate System” entry from the “Plots” menu, and fill in the dialog as shown on Figure 14. Note that the fields for location of ticks are left empty, which means that ticks will appear at every integer. This will define the viewing rectangle, and create the x and y axes, with 5 pt long ticks at every integer (Figure 15).

After creating the coordinate system, it should be selected. If it is not, select it using the “select” tool. The next step will be creating a coordinate grid. Select the “Coordinate grid” entry from the “Plots” menu. The Coordinate grid dialog

Figure 14: The dialog box for coordinate system

will appear, with information already filled (Figure 16). IpePlots automatically filled in this information based on the data you entered in the “Coordinate system” dialog. If you are happy with these choices, you can just click the OK button. The coordinate system shown in Figure 17 will be created. It will be created with the currently active line style. For our purpose, we want to change this into dashed style using the IPE properties panel.

Now we need to create the first part of the graph: $y = x + 5$ for $x < -2$. Make sure that either the coordinate system or the coordinate grid is selected. Then choose “Function plot” from the “Plots” menu. As before, the dialog is partially filled based on the information that you entered in the previous dialogs. Fill in the remaining entries as shown on Figure 18. Note that since we are plotting a line segment, we only need to use 2 points, and we do not want to use cubic splines¹. You can see the result in Figure 19.

The next part of the plot is the parabolic arc $y = x^2 - 1$ for $-2 \leq x < 1$. First select either the coordinate system or the coordinate grid in which the parabola should be placed. Then choose “Function plot” from the “Plots” ipelet menu. The dialog box that will open will contain the values that you entered when plotting the first part. You need to edit those as shown in Figure 20. The entries you need to change are the equation for y and the domain for x . Also, since we are no longer plotting a straight line segment, you probably want to increase the number of plot points. You may also want to select cubic spline approximation, which will result in a smoother plot². Figure 21 shows the plot with the first two parts present.

In a similar way, we create the third part of the plot. Using IPE marks with vertex snapping, you can place full and empty circles at the ends of the parts of the graph to indicate whether the endpoints are included or not. You can also change the style of the plot curves to “fat” or “ultrafat” to make them stand out against the coordinate grid. The finished plot is at Figure 22.

¹The figure will look perfectly fine with larger number of points, and of course when using cubic spline to approximate a linear function, we get the correct linear function, however, it would make IpePlots generate more complicated code, which would then result in a larger file.

²Since we are plotting a parabola, using cubic splines with 4 points would work perfectly fine.

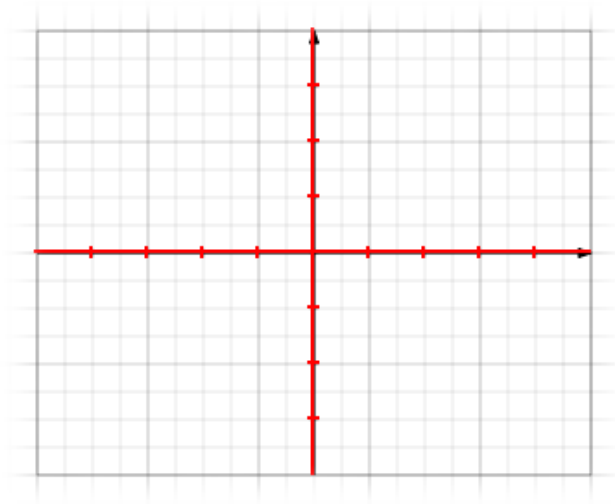


Figure 15: The coordinate system created by the dialog box in Figure 14

Set coordinates for viewport:

from x= to x=

from y= to y=

Locations of vertical grid lines:

Locations of horizontal grid lines:

Figure 16: The dialog for creation of a coordinate grid

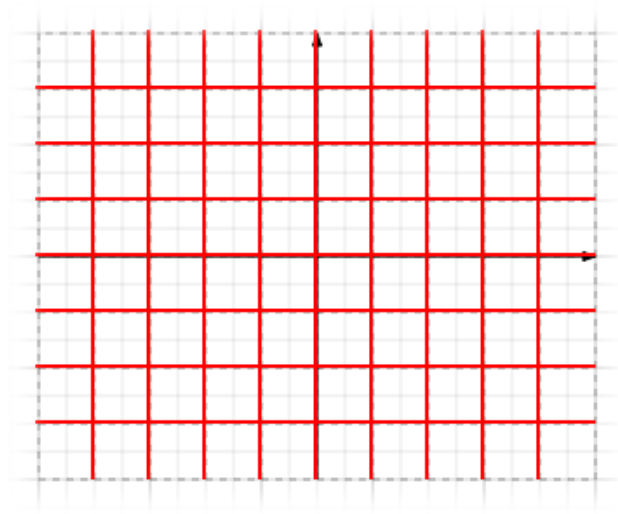


Figure 17: The coordinate grid created by the dialog box in Figure 16

Enter y as a function of x

y=

Set the domain for x:

from: to:

Set coordinates for viewport:

from x= to x=

from y= to y=

number of points

☐ use cubic splines

Figure 18: The dialog box that will create the first part of the plot: $y = x + 5$ for $x < -2$

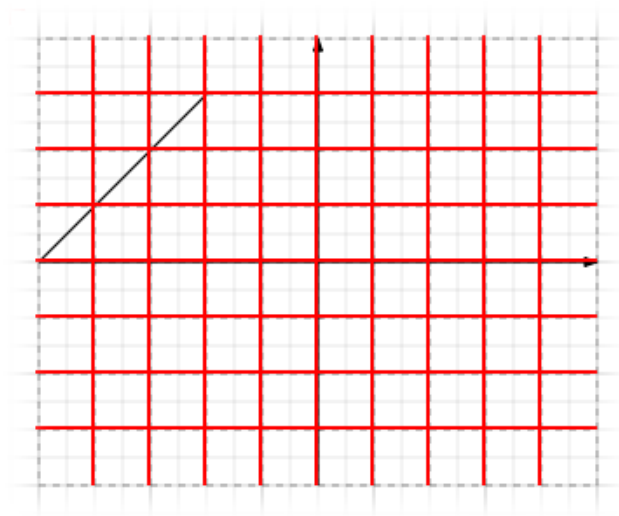


Figure 19: The first part of the plot of the piecewise function

Enter y as a function of x

y=

Set the domain for x:

from: to:

Set coordinates for viewport:

from x= to x=

from y= to y=

number of points

☒ use cubic splines

Figure 20: The dialog box for the second part of the plot of the piecewise function f

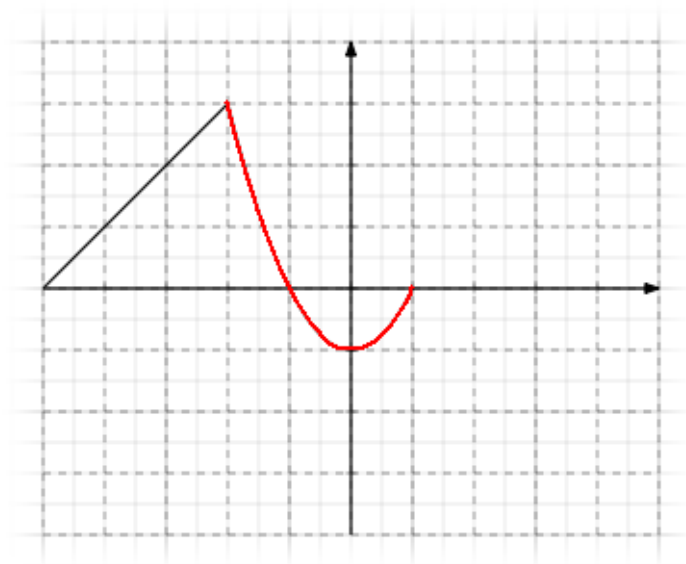


Figure 21: Plot of the first two parts of the piecewise function f

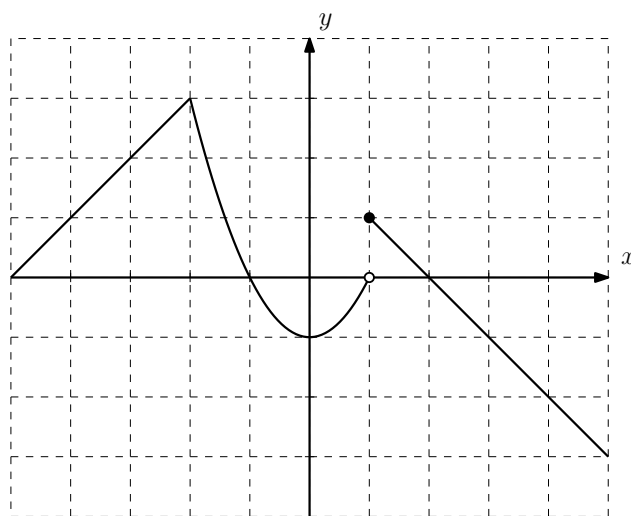


Figure 22: Graph of the piece wise function f .

4.2 Function with Vertical Asymptote

As the next example, we will plot the function

$$g(x) = \frac{1}{1-x^2}.$$

The function is undefined at ± 1 and has vertical asymptotes there. The IpePlots ipelet is not smart enough to figure that out, and it will not be able to plot the function properly. It is our responsibility to choose the domain of the plot properly. We will plot the function for x between -4 and 4 , and y between -3 and 3 . Solving the equation

$$\frac{1}{1-x^2} = -3$$

will give us $x = \pm\sqrt{4/3}$, solving the equation

$$\frac{1}{1-x^2} = 3$$

results in $x = \pm\sqrt{2/3}$. We will plot the function on three intervals: $-4 \leq x \leq -\sqrt{4/3}$, $-\sqrt{2/3} \leq x \leq \sqrt{2/3}$ and $\sqrt{4/3} \leq x \leq 4$.

Enter y as a function of x

y=

Set the domain for x:

from: to:

Set coordinates for viewport:

from x= to x=

from y= to y=

number of points

☒ use cubic splines

Figure 23: The dialog to create the first part of the graph of g .

First we will create a coordinate system. Then choose “Function plot” from the “Plots” menu, and fill in the dialog as shown on Figure 23. Note that it is possible to use expressions like `-sqrt(4/3)` in the “from” and “to” fields of the dialog. Figure 24 shows the dialog for creation of the second part of the graph. The third part can be created in a similar way. Figure 25 shows the resulting plot.

Enter y as a function of x

y=

Set the domain for x:

from: to:

Set coordinates for viewport:

from x= to x=

from y= to y=

number of points

☒ use cubic splines

Figure 24: The dialog to create the second part of the graph of g .

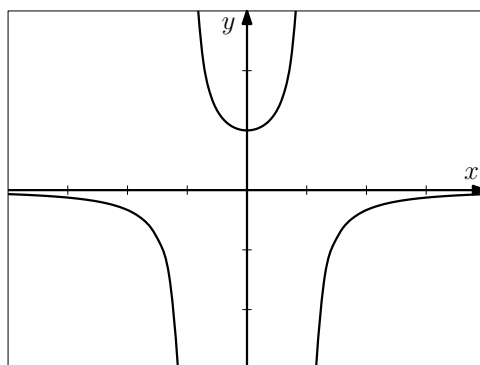


Figure 25: The final plot of the function g .

4.3 Trigonometric Plot

As a final example, we will plot a trigonometric function, with ticks on the horizontal axis at multiples of $\pi/2$. We will start by creating a coordinate system, for x between -2π and 2π , and y between -2.2 and 2.2 . See Figure 26. The most interesting part is the expression entered in the “Location of x -ticks”

Set coordinates for viewport:

from $x=$ to $x=$

from $y=$ to $y=$

Size of x -ticks (in pt): Size of y -ticks (in pt):

Locations of x -ticks:

Locations of y -ticks:

Figure 26: The dialog to create the coordinate system for a trigonometric plot.

field: `range(-2*pi,2*pi,pi/2)`. This will create ticks uniformly distributed between -2π and 2π , with distance $\pi/2$ between consecutive ticks.

Enter y as a function of x

$y=$

Set the domain for x :

from: to:

Set coordinates for viewport:

from $x=$ to $x=$

from $y=$ to $y=$

number of points

☒ use cubic splines

Figure 27: The dialog to create the coordinate system for a trigonometric plot.

Next we will create the actual plot using the “Function plot” item from the “Plots” menu, as shown in Figure 27. Finally, we change the line thickness of the graph to “ultrafat”, and add some legend to the ticks on both horizontal and vertical axis. The resulting plot is shown in Figure 28

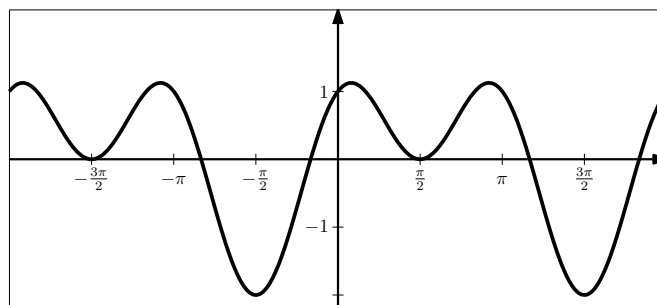


Figure 28: The final plot of the trigonometric function $\sin(x) + \cos(2x)$.

References

- [1] Otfried Cheong. *Ipe*. 2024. URL: <https://ipe.otfried.org/ipe-manual.pdf>.
- [2] Otfried Cheong. *The Ipe extensible drawing editor*. Version 7.2.29. June 26, 2025. URL: <https://ipe.otfried.org>.